

Claude CLI Integration with Tavro

1. Objective

The objective of this implementation is to integrate Claude Code capabilities with the Tavro platform so that users can generate, modify, deploy, and manage agent source code directly from Tavro. The integration uses the Anthropic API and Claude CLI, while Tavro acts as an orchestration layer that connects agent definitions, code-generation workflows, Git-based source control, and Azure AI Foundry deployments.

The integration introduces support for Code-driven agents, enabling users to generate and maintain production-ready Python agent implementations through the Tavro Agent Playground.

2. Prerequisites

Before starting the integration, the following prerequisites are required:

2.1 Anthropic API Key

- A Personal API key — create one at console.anthropic.com → API Keys.
- The key will be set as `ANTHROPIC_API_KEY` in `.env`.

2.2 Azure AI Foundry

Azure AI Foundry is required only when deploying generated agents using the Deploy functionality.

Required prerequisites:

- Active Azure subscription.
- Azure AI Foundry resource.
- Azure AI Foundry project with Hosted Agents enabled.
- Appropriate permissions to create and manage hosted agents.

Reference Docs:

https://learn.microsoft.com/en-us/azure/foundry/agents/concepts/hosted-agent-permissions?utm_source=chatgpt.com

2.3 Git Repository

Git integration is required only when using the Publish to Git functionality.

Required prerequisites:

- GitHub repository for storing generated agent code.
- GitHub Classic Personal Access Token (PAT) with `repo` scope permissions.

2.4 Tavro Application

The following components are required:

- Tavro source code downloaded from the Tavro repository.
- Docker and Docker Compose installed.
- Git installed for version control.
- Basic understanding of Docker container networking and environment configuration.

3. Integration Approach

The integration uses a backend orchestration architecture. Claude capabilities are integrated directly into the Tavro platform through the Tavro API service.

Tavro acts as a bridge between the Anthropic platform, Git repositories, Azure AI Foundry, and Tavro workflows. Users interact with Claude capabilities through the Tavro user interface, while all communication with Anthropic services is managed by the backend.

The Tavro API service (`tavro-api`) uses the Anthropic API and Claude CLI to process user requests and interact with Anthropic services.

The integration supports:

- Interaction with Claude through the Tavro platform.
- Integration with Git repositories.
- Integration with Azure AI Foundry deployment workflows.
- Real-time streaming of responses and status updates to the user interface.

Claude CLI is installed as part of the Tavro deployment and is invoked through backend workflows. No separate Claude installation is required for end users.

4. Integration Steps

4.1 Clone the Tavro Repository

Clone the Tavro source code from:

```
git clone https://github.com/TavroOrg/tavro.git
cd tavro
```

4.2 Environment Configuration

Open .env and provide appropriate values.

Anthropic Configuration:

```
#AI-Keys
```

```
ANTHROPIC_API_KEY=
```

Git Publishing Configuration:

```
# — Agent Playground — Publish to Git
```

```
# GitHub repo where generated agent code is published from the  
Playground Code tab.
```

```
# Token requires `repo` scope.
```

```
GIT_PUBLISH_REPO_URL=
```

```
GIT_PUBLISH_TOKEN=
```

```
GIT_PUBLISH_BRANCH=
```

Azure AI Foundry Configuration :

```
AZURE_AI_FOUNDRY_ENDPOINT=
```

```
AZURE_AI_FOUNDRY_KEY=AZURE_AI_FOUNDRY_API_VERSION=2024-02-15-preview
```

```
AZURE_AI_FOUNDRY_AGENT_API_VERSION=v1
```

```
# Optional. If omitted, the selected playground model is used as the  
Azure deployment name.
```

```
AZURE_AI_FOUNDRY_DEPLOYMENT=gpt-4o
```

```
AZURE_AI_FOUNDRY_CLIENT_ID=
```

```
AZURE_AI_FOUNDRY_TENANT_ID=
```

```
AZURE_AI_FOUNDRY_CLIENT_SECRET=
```

```
# Hosted-agent deployments — set to the resource/project that has
Hosted Agents enabled.

# If omitted, AZURE_AI_FOUNDRY_ENDPOINT is used as the fallback.

AZURE_AI_FOUNDRY_HOSTED_ENDPOINT=
```

4.3 Build and Start the Environment

Build and start all Tavro services:

`docker compose up -d --build`

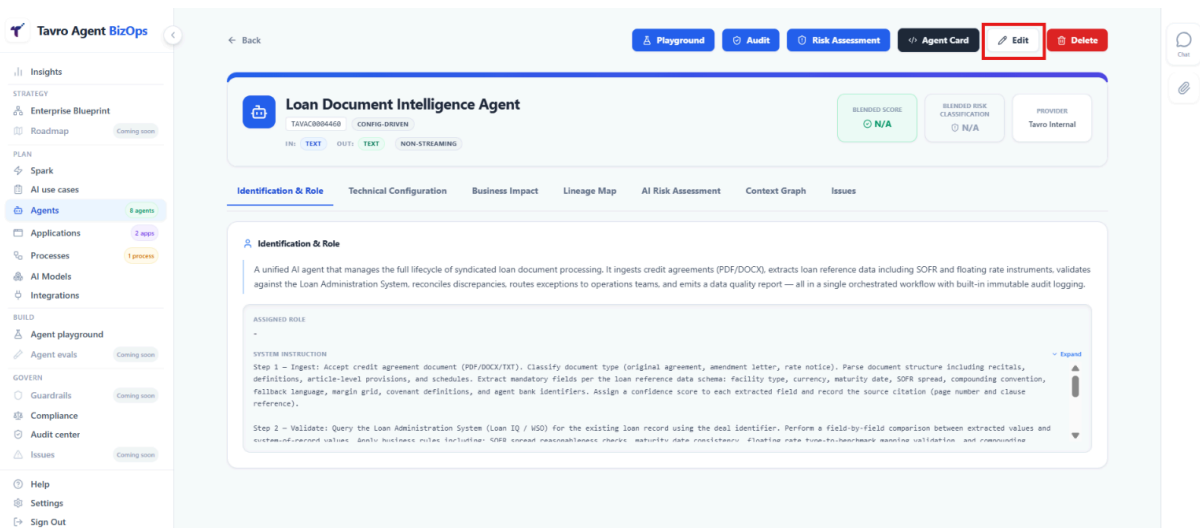
Verify that:

- `tavro-api` starts successfully.
- `tavro-app` starts successfully.
- The Tavro UI loads without errors.
- No containers exit unexpectedly.

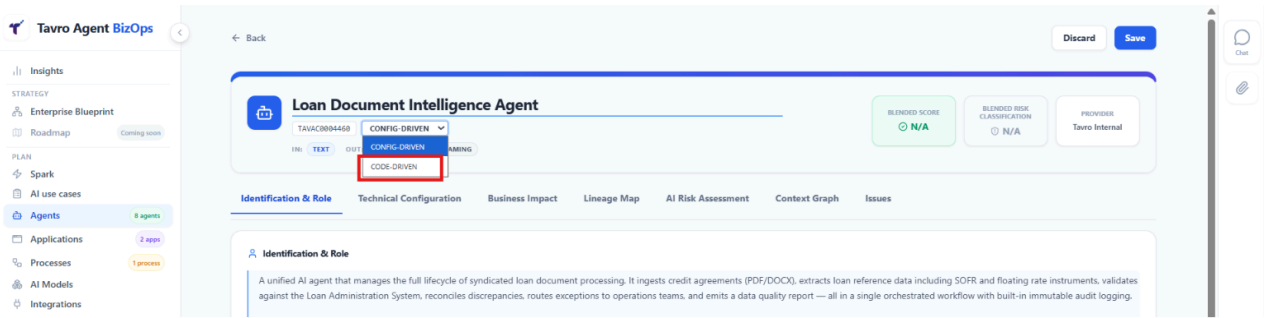
4.4 Configure a Code-Driven Agent

To configure an agent:

- Open the Agent Catalog.
- Select the required agent.
- Click Edit.



- Locate the Agent Type field.
- Select Code-driven from the dropdown list.
- Click Save.



Once the changes are saved, the Code tab becomes available in the Agent Playground for the selected agent.

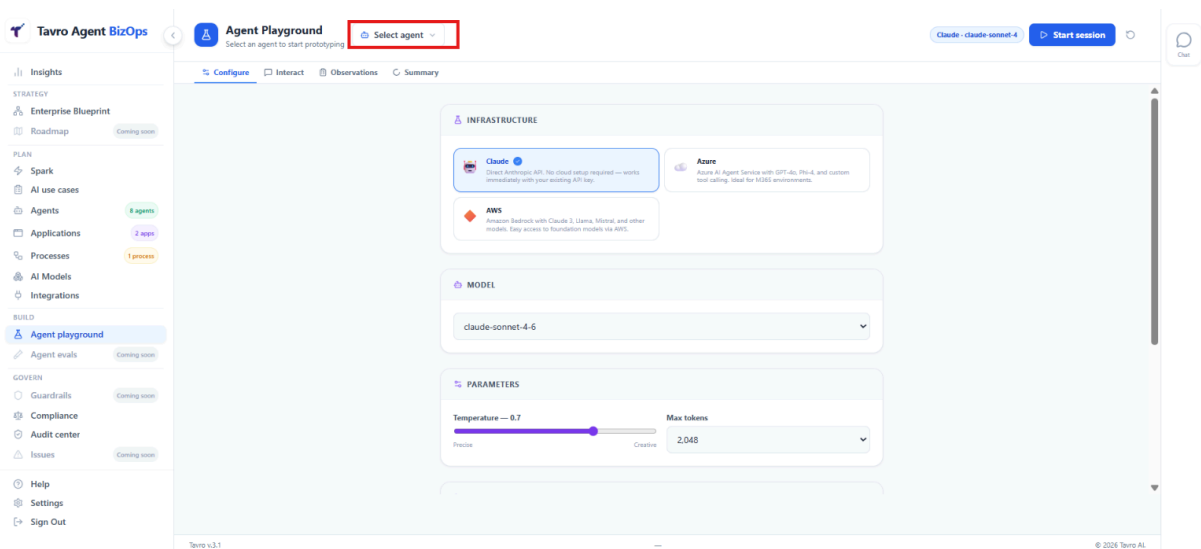
Note: Agents without an Agent Type specified are treated as **Config-driven** by default and will not display the Code tab.

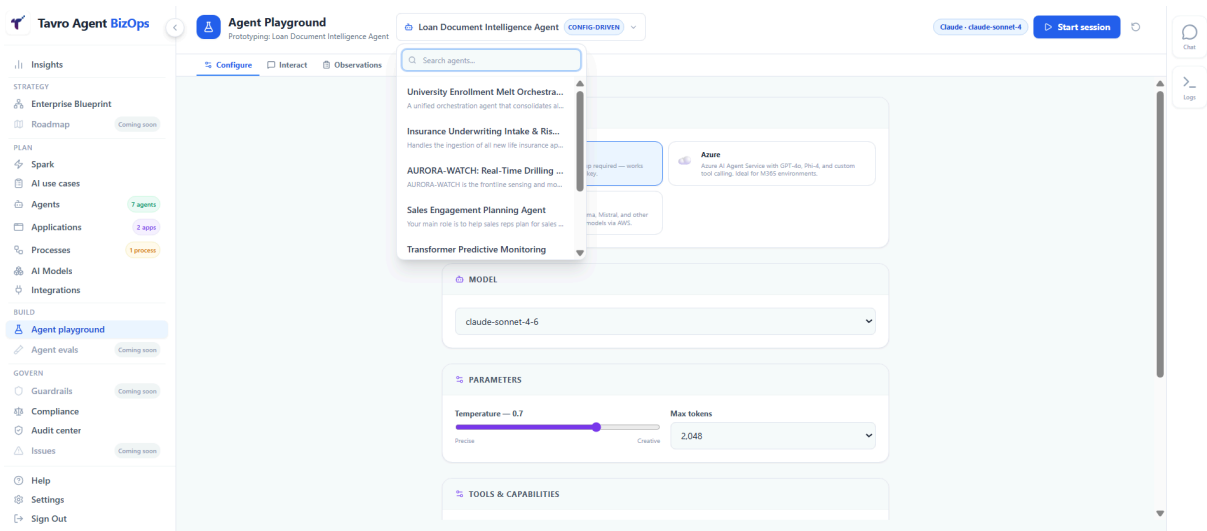
5. Agent Playground

5.1 Select a Code-Driven Agent

Open the Agent Playground.

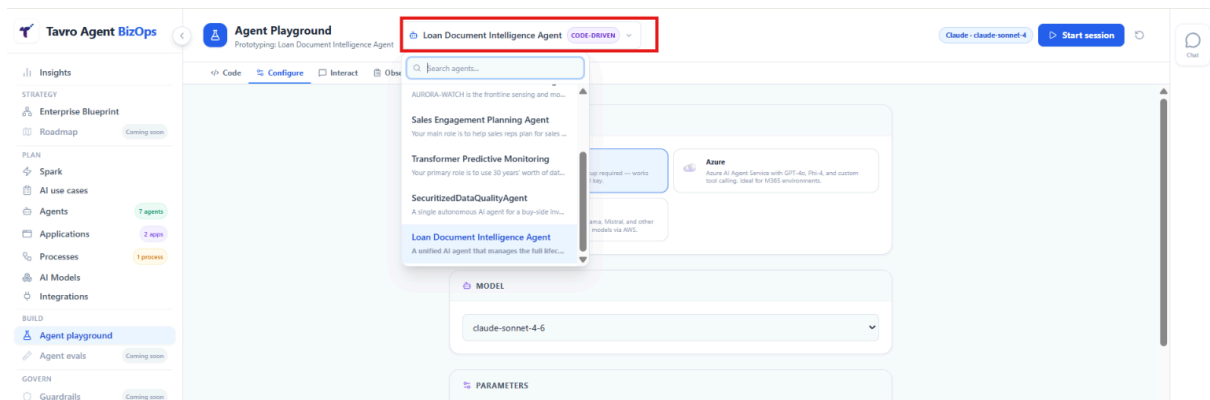
Click the Select Agent dropdown.





Choose an agent that is configured as **Code-driven**.

The selected agent is loaded into the Agent Playground.

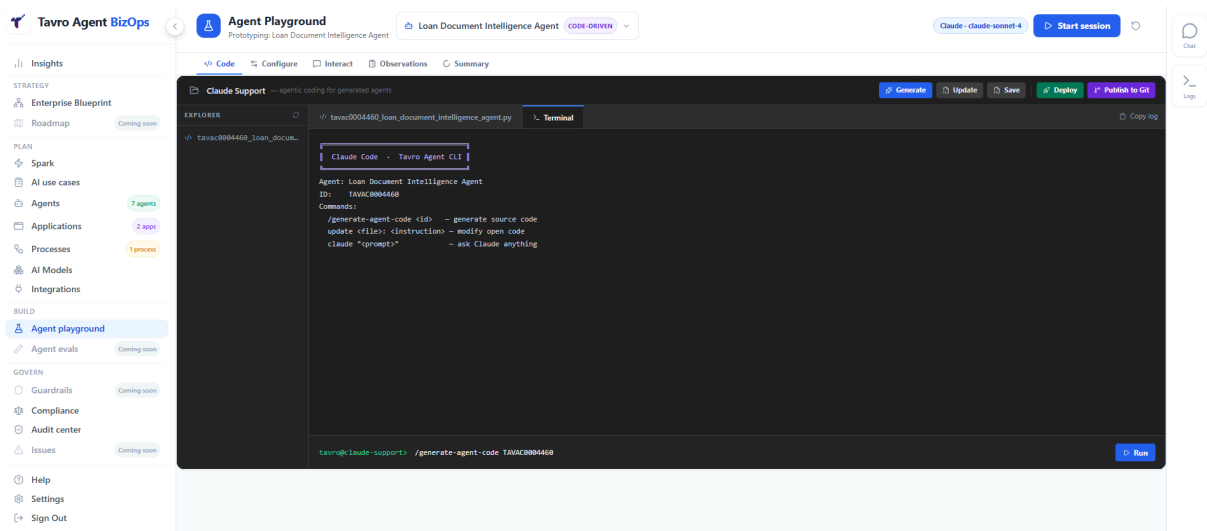


Navigate to the **Code** tab to access code generation and editing features.

5.2 Generate Agent Code

To generate code for the selected agent:

- Open the **Code** tab.
- Click **Generate**.



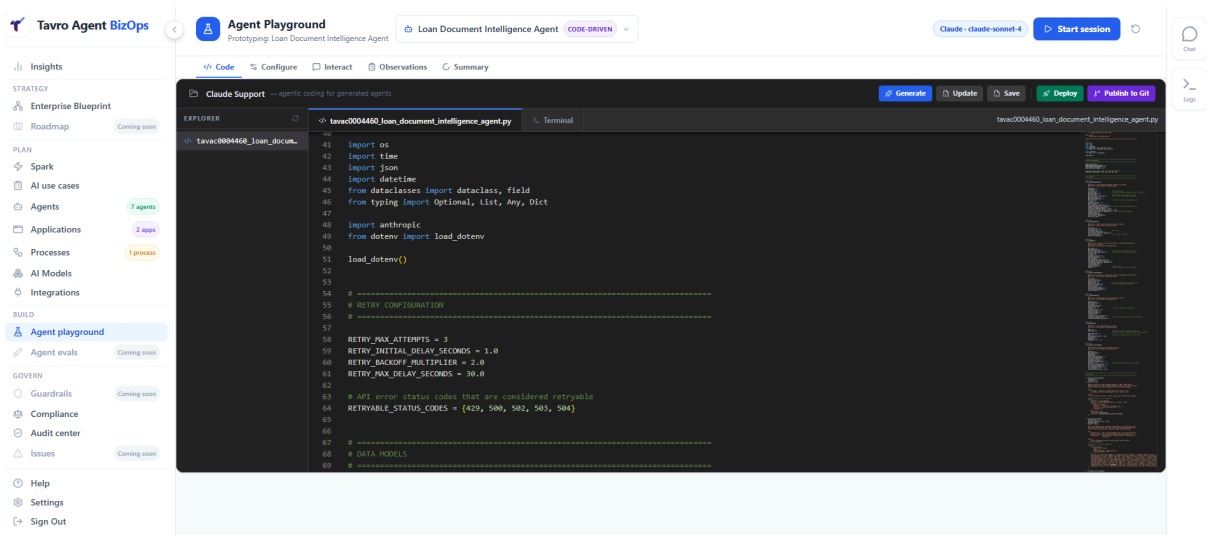
Alternatively, execute the following command in the terminal:

```
/generate-agent-code <agent_id>
```

Example:

```
/generate-agent-code TAVAC0004434
```

The generated Python code is streamed directly into the editor.



5.3 Update Existing Code

Generated code can be modified using natural-language instructions.

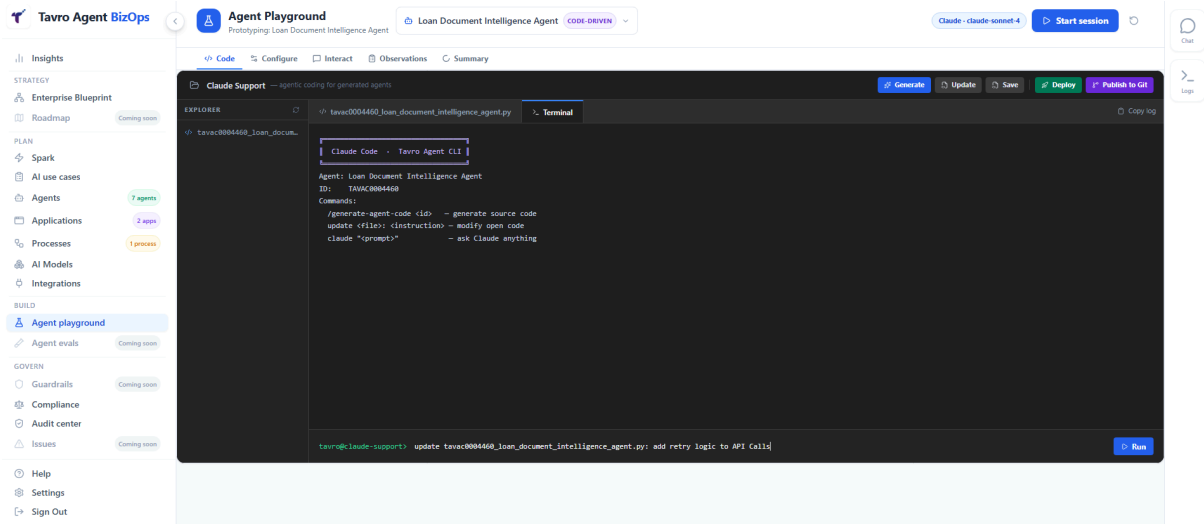
To update code:

- Open the generated file in the Monaco editor.

- Click **Update**.
- Enter the required modification instruction.
- Submit the request.

Example:

update agent.py: add retry logic to API calls

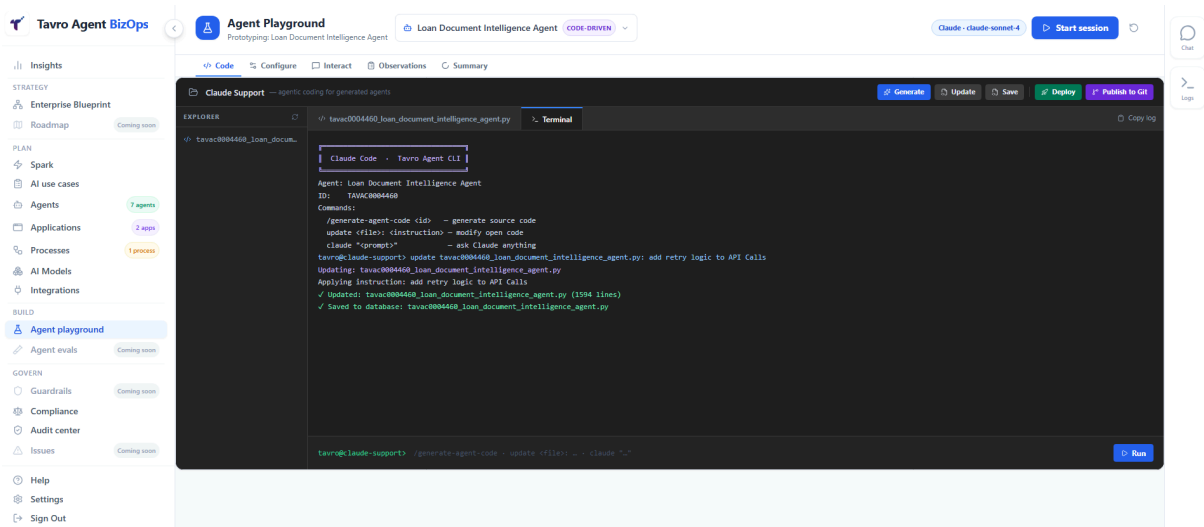


Claude analyzes the existing code and applies the requested changes. The updated code is streamed back into the editor.

5.4 Save Code

To save the current editor contents:

- Review the generated or modified code.
- Click **Save**.



The code is stored in:

- `core.agent_generated_code`

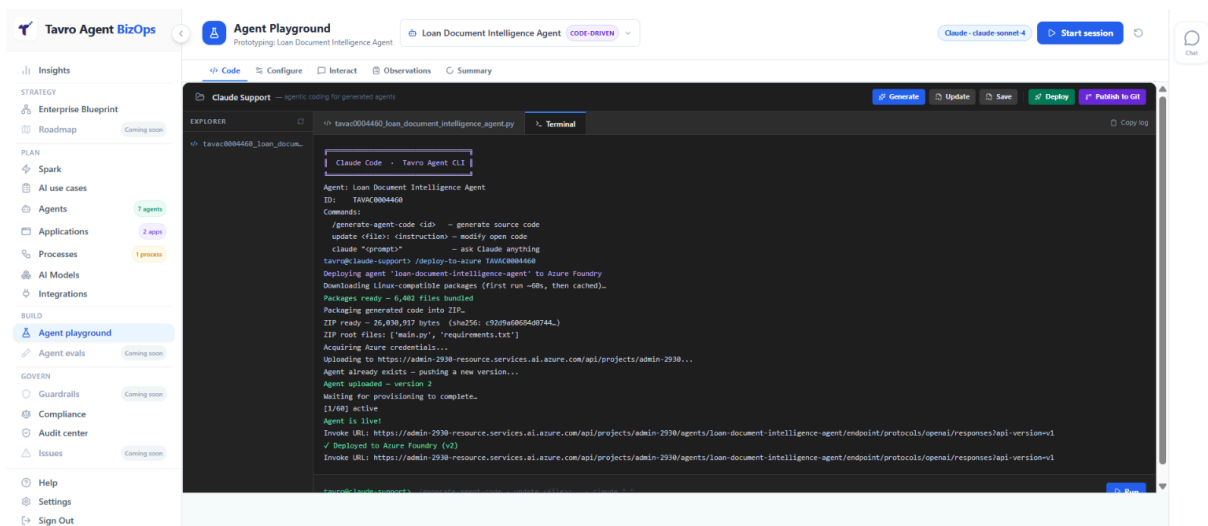
Saved code is automatically reloaded when the agent is reopened or the page is refreshed.

5.5 Deploy to Azure AI Foundry

Generated agents can be deployed as Azure AI Foundry Hosted Agents.

To deploy:

- Save the generated code.
- Click **Deploy**.
- Monitor deployment progress in the terminal window.
- Wait for deployment completion.



Alternatively, use the terminal command:

`/deploy-to-azure <agent_id>`

Example:

`/ deploy-to-azure TAVAC0004434`

Upon successful deployment, the agent becomes available within the configured Azure AI Foundry project.

Name	Version	Type	Created on ↓	Description
loan-document-intelligence-agent	2	hosted	6/19/26, 5:58:07 PM	Tavro-deployed agent: loan-document-intelligence-agent
bug-reporting-agent	1	hosted	6/19/26, 4:43:38 PM	Tavro-deployed agent: bug-reporting-agent
securtizeddataqualityagent	1	hosted	6/19/26, 11:59:29 AM	Tavro-deployed agent: securtizeddataqualityagent
ci-cd-automation-agent	1	hosted	6/18/26, 1:20:28 PM	Tavro-deployed agent: ci-cd-automation-agent
sales-engagemnt-planning-agent	3	hosted	6/18/26, 9:27:21 AM	Tavro-deployed agent: sales-engagemnt-planning-agent
transformer-predictive-monitoring	1	hosted	6/17/26, 2:49:23 PM	Tavro-deployed agent: transformer-predictive-monitoring

5.6 Publish to Git

Generated code can be published to a GitHub repository.

To publish:

- Save the generated code.
- Click **Publish to Git**.
- Tavro commits the current code to the configured repository.

The screenshot shows the Tavro Agent Playground interface. On the left is a sidebar with navigation options like Insights, Strategy, Plan, Build, and Govern. The main area displays the 'Loan Document Intelligence Agent' with a 'CODE-DRIVEN' status. A terminal window is open, showing the following code:

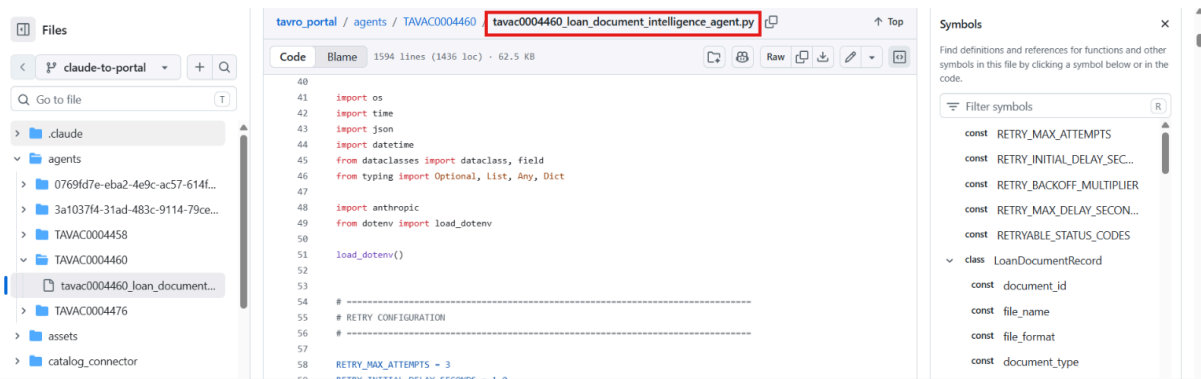
```

Claude Code - Tavro Agent CLI
Agent: Loan Document Intelligence Agent
ID: TAVAC0004460
Commands:
/generate-agent-code cld - generate source code
update (file): <instructions> - modify open code
claude "prompt" - ask Claude anything
tavro@claude-support> publish-to-git tavac0004460_loan_document_intelligence_agent.py
✓ Published to git
Repo: TavroOrg/tavro_portal
Branch: claude-to-portal
Path: agents/TAVAC0004460/tavac0004460_loan_document_intelligence_agent.py
URL: https://github.com/TavroOrg/tavro_portal/blob/claude-to-portal/agents/TAVAC0004460/tavac0004460_loan_document_intelligence_agent.py

tavro@claude-support> /generate-agent-code - update (file) - - claude "..."
  
```

Published files are stored under:

agents/<agent_id>/<filename>.py



The repository is automatically updated with the latest version of the generated code.

6. Validation

The integration is considered successful when the following validations pass:

- A Code-driven agent displays the Code tab in the Agent Playground.
- Agent code is successfully generated using the Generate action.
- Generated code is displayed in the editor.
- Code updates are successfully applied using natural-language instructions.
- Save persists code to the core .agent_generated_code table.
- Refreshing the page reloads previously saved code.
- Publish to Git creates or updates files in the configured GitHub repository.
- Deploy successfully creates an Azure AI Foundry hosted agent.

Successful completion of these checks confirms that the Claude CLI integration has been configured and is functioning correctly within the Tavro platform.